Rheinische Friedrich-Wilhelms-Universität Bonn

Bachelorstudiengang Informatik

KORREKTURHINWEISE – NICHT ZUR VERTEILUNG

Prüfungsklausur

Semester: WS ☑ SS □ *2014/15*	Versuch: 1. □ 2. ☑
Modulbezeichnung: ***Systemnahe Programmierung***	
Modulnummer: *BA-INF 034/129*	Prüfungsdatum: 25. *März 2015*
1. Prüfer: *Prof. Dr. Matthew Smith*	2. Prüfer: *Dr. Matthias Frank*

vom Prüfling auszufüllen:

Name:	Matrikel-Nr:
Vorname:	Semester:
Studiengang Bachelor Informatik ☐ Lehramt ☐ Informatik als Nebenfach ☐	

vom Prüfer auszufüllen:

Bewertung:

*Noten und Notenwerte: **sehr gut (1,0; 1,3) – gut (1,7; 2,0; 2,3) – befriedigend (2,7; 3,0; 3,3) – ausreichend (3,7; 4,0) – nicht ausreichend (5,0)**

Datum: 25. *März 2015*

KORREKTURHINWEISE – NICHT ZUR VERTEILUNG

---------- ----------

Unterschrift des 1. Prüfers Unterschrift des 2. Prüfers

Prof. Dr. Matthew Smith
Dr. Matthias Frank

Bonn, den 25. März 2015

Hinweise zur Modulprüfung zu „Systemnahe Programmierung“ (BA-INF 034/129)

Wintersemester 2014/2015
9:30 Uhr – 11:00 Uhr

Hinweise (GENAU DURCHLESEN!)

- Neben Papier und Schreibutensilien sind **keine weiteren Hilfsmittel** erlaubt. Verwenden Sie nur dokumentenechte Stifte (z.B. keine Bleistifte). Verwenden Sie keine roten Stifte.
- Vergessen Sie nicht, Ihren Namen und die Matrikelnummer auf *jedes* Blatt zu schreiben. Blätter ohne diese Angaben werden nicht gewertet.
- Schreiben Sie Ihre Lösungen auf die Aufgabenblätter möglichst in die dafür vorgesehenen Felder. Sie können auch die Rückseiten verwenden. Weiteres Schreibpapier kann von den Betreuern angefordert werden. Benutzen Sie kein mitgebrachtes Papier.
- Bitte schreiben Sie in Ihrem eigenen Interesse deutlich. Für nicht lesbare Lösungen können wir keine Punkte vergeben.
- Klausurblätter dürfen nicht voneinander getrennt werden.
- Werden mehrere unterschiedliche Lösungen für eine Aufgabe abgegeben, so wird die Aufgabe nicht gewertet.
- Im Fall von Täuschungsversuchen wird die Klausur sofort mit 0 Punkten bewertet. Eine Vorwarnung erfolgt nicht.
- In der Klausur können Sie 90 Punkte erhalten. Achten Sie darauf, dass Sie im Schnitt nicht viel mehr als 1 Minute pro Punkt zur Verfügung haben. Mit 50% dieser Gesamtpunktzahl haben Sie die Klausur sicher bestanden.

Viel Erfolg!

1	**2**	**3**	**4**	**5**	**6**	**7**	8	**Gesamt**
Multiple Choice	C/Sockets	Assembler	PThreads	Go	IPC	Byte Order	Sockets UDP	
9	12	12	15	10	10	12	10	90

Aufgabe 1 : (Multiple-Choice, 9 Punkte)

Bei den folgenden Multiple-Choice-Fragen wird für jedes richtig gesetzte Kreuz 1 Punkt vergeben. Bei falschen Antworten werden 0 Punkte vergeben.

	Aussage	Richtig	Falsch
1	Ein TCP-Client bereitet sich durch die Abfolge von socket() und connect() für die Kommunikation mit einem TCP-Server vor.	☒	☐
2	Beim sog. „select loop“ für einen TCP-Server bedient ein einziger Prozess in einer endlos laufenden Schleife mit select() mehrere Sockets für alle aktiven Verbindungen. Eingehende Verbindungsaufbauwünsche müssen separat überwacht werden.	☐	☒
3	Kommunizieren zwei Rechner mit derselben Byte-Order über ein Cross-Link-Kabel, darf nicht auf die Network-Byte-Order-Umwandlung verzichtet werden.	☐	☒
4	Bei allen der hier genannten Assembler-Befehle ADD, SUB, CMP, AND, OR, XOR (alle mit 2 Operanden) wird in einem der beiden Operanden das Ergebnis abgelegt (sog. „Überdeckung“).	☐	☒
5	Ein Zombi-Prozess ist ein Kindprozess, der noch läuft obwohl sein Vater beendet worden ist.	☐	☒
6	FIFOs (Named Pipes) sind nach einem Neustart des Kernels weiterhin im Dateisystem zu finden.	☐	☒
7	In Go gibt es dank dem internen Scheduler keine Race-Conditions.	☐	☒
8	In Go werden ALLE Variablen und Datenstrukturen direkt auf dem Stack gespeichert.	☐	☒
9	Funktionen werden mit dem `go` Keyword asynchron als Goroutine ausgeführt.	☒	☐

Sollten Sie der Meinung sein, dass eine (oder mehrere) der obigen Fragen missverständlich mit „Richtig“ oder „Falsch“ zu beurteilen sind, dann können Sie unten Ihre gewählte Antwort kurz begründen. Bitte achten Sie dabei selbst auf Ihre Zeiteinteilung: Es macht keinen Sinn, viele (oder gar alle) der Multiple-Choice-Fragen durch ausführlichen Text zu begründen. Es gibt nur für die oben gewählten Antworten Punkte, nicht für Ihre Erläuterungen.

Nr.	Ihre Erläuterung

Aufgabe 2: (C Socket-Programmierung, 12 Punkte [6 x 2])

Aus der Vorlesung ist Ihnen die folgende TCP Client-Server-Anwendung bekannt. Der Client erhält den Host-Namen des Servers als Kommandozeilenparameter und liest einen String von der Standardeingabe ein. Dieser soll an den Server geschickt werden, der den String in Großbuchstaben umwandeln und an den Client zurücksenden soll. Der Client gibt den empfangenen String auf der Standardausgabe aus.
In das Programm haben sich an den 6 markierten Stellen Fehler eingeschlichen. Schreiben Sie jeweils in den Kasten „Korrektur“ eine korrigierte Version der entsprechenden Zeile. Kennzeichnen Sie das Löschen einer Zeile durch das Wort „Löschen“ an der entsprechenden Stelle im Korrekturkasten.

Beachten Sie, das Client und Server zusammenarbeiten sollen!

Korrektur – TCP Server:

(1) printf("Local Port: %d\n", ntohs(myaddr.sin_port));

(2)
```
if ((newfd = accept(li, (struct sockaddr *)&remaddr,
                    &addrlen)) >= 0) {
```

(3) write(newfd, buf, nbytes, …);

Korrektur – TCP Client:

(4) serv_addr.sin_family = AF_INET;

(5) if ((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) return 1;

(6) if ((connect(fd, (struct sockaddr *)&serv_addr,
sizeof(struct sockaddr_in))) < 0) return 1;

Aufgabe 3: (Assembler – Programmablauf analysieren, 12 Punkte [3+3+2+4])

Gegeben ist das im Folgenden dargestellte Assembler-Programm. Aus Gründen der Übersichtlichkeit ist das Programm auf diese und die folgenden Seiten verteilt.

Hauptroutine des Assemblerprogramms:

```
.text

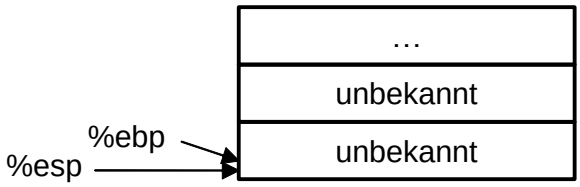
.global _start

_start:

  # call funktion(3, 9)
  pushl  $9
  pushl  $3
  call   funktion
  # Parameter wieder vom Stack nehmen
  addl   $8,%esp

  # exit process
  movl   $1,%eax
  movl   $0,%ebx
  int    $0x80
```

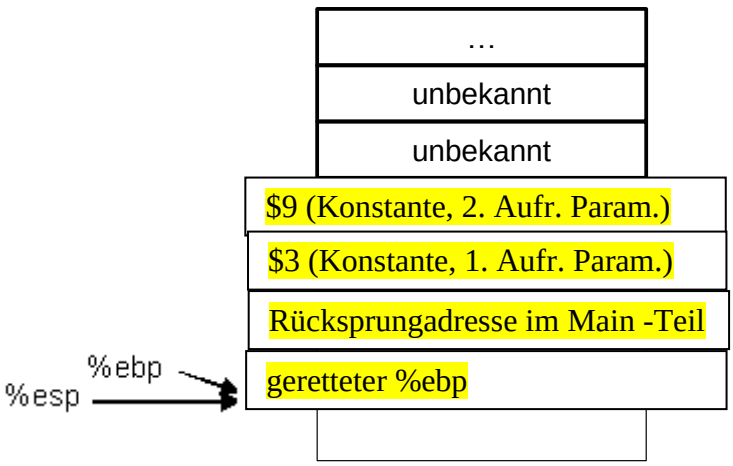
Stack-Aussehen nach Label "_start"



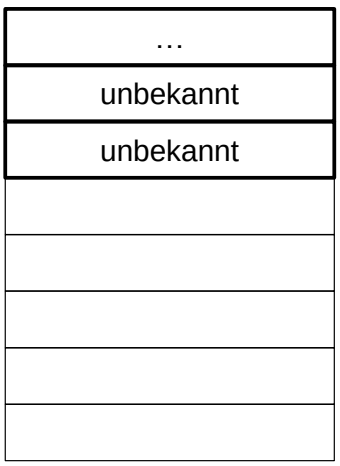
a) (3 Punkte) Beim Eintritt in die Hauptroutine des Assemblerprogramms hat der Stack das oben gezeigte Aussehen (Zeitpunkt vor der Ausführung des ersten Befehls nach dem Label „_start“).

Ergänzen Sie nun den Stack (Belegung der Zellen mit Konstanten bzw. kurze Erklärung des Inhaltes), wie er nach dem Betreten der Unterroutine „funktion“ (siehe übernächste Seite) beim Erreichen des Labels „Zeitpunkt_1“ aussieht. Kennzeichnen Sie auch die dann aktuelle Position des Basepointers %ebp und des Stackpointers %esp.

Platz für Ihre Lösung:
(bitte streichen Sie eine ungültige Lösung deutlich)



Reserve-Platz für Ihre Lösung



b) (3 Punkte) Betrachten Sie nun die Unterroutine „funktion“ auf der nächsten Seite genauer und verfolgen Sie den weiteren Ablauf.

Ergänzen Sie jetzt den Stack (Belegung der Zellen mit Konstanten bzw. kurze Erklärung des Inhaltes), wie er nach dem Erreichen des Labels „Zeitpunkt_2“ aussieht. Kennzeichnen Sie auch hier wieder die aktuelle Position des Basepointers %ebp und des Stackpointers %esp.

Ergänzen Sie Ihre Lösung an der dafür vorgesehenen Stelle auf der nächsten Seite.

c) (2Punkte) Welcher (fatale) Fehler wird von der Unterroutine hervorgerufen?

Lösung:

Mit dem Befehl movl %ebx, 4(%ebp) wird die Rücksprungadresse überschrieben!

d) (4 Punkte) Verändern Sie das gegebene Programm in der Weise, dass der Fehler aus c) nicht mehr auftritt und dass das Programm das Ergebnis der letzten arithmetischen Operation gemäß der in der Vorlesung besprochenen „cdecl Calling Convention“ an den Aufrufer als Rückgabewert zurückgibt.

Lösung:
(Links: ursprüngliche Sourcezeile Rechts: veränderte Sourcezeile)

__addl %eax, %ebx_____	___addl %ebx, %eax_____
__movl %ebx, 4(%ebp)_____	___(komplett streichen)_____
_____	_____

Unterroutine „funktion“ des Assemblerprogramms:

```
# int funktion(int a, b)
funktion:

  # set up stack frame
  pushl %ebp
  movl  %esp,%ebp

Zeitpunkt_1:

  # reserviere Platz für 2 lokale Variablen
  subl  $8,%esp


  movl  8(%ebp), %eax
  movl  %eax, -4(%ebp)

  movl  12(%ebp), %ebx
  movl  %ebx, -8(%ebp)

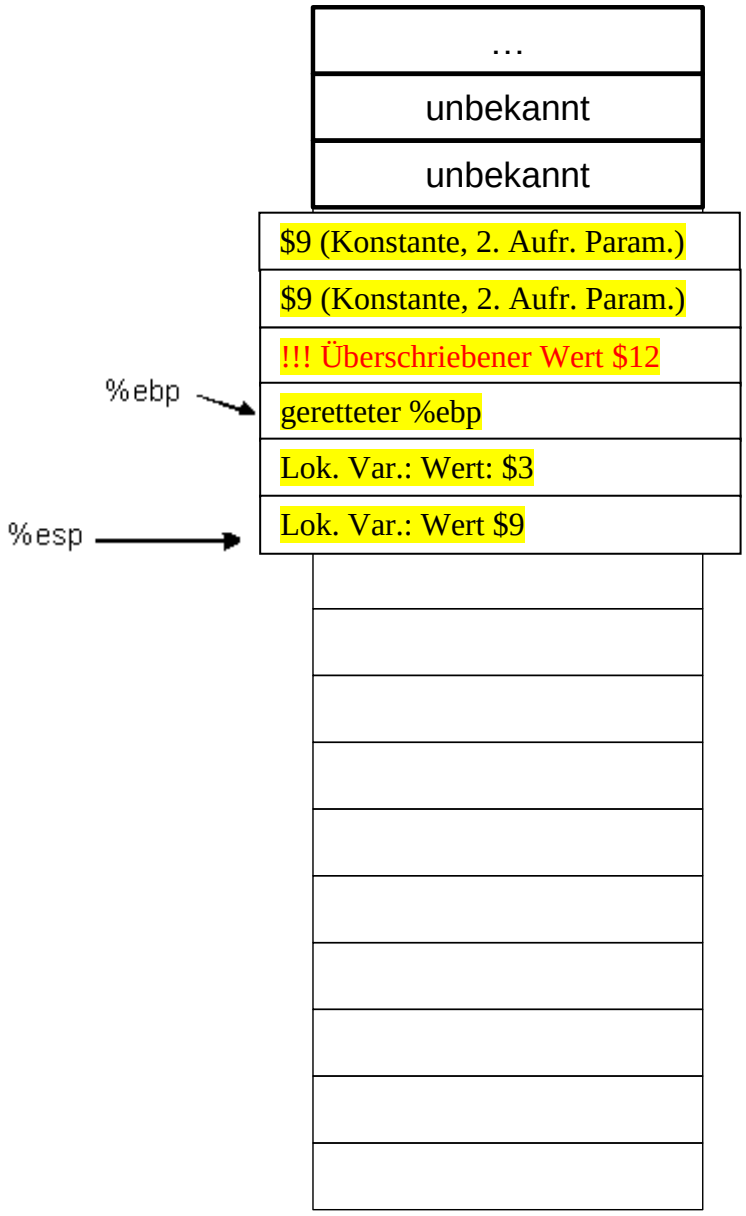
  addl  %eax, %ebx
  movl  %ebx, 4(%ebp)


Zeitpunkt_2:

  # reconstruct stack- and base-pointer
  leave

  # return
  ret
```

Platz für Ihre Lösung Teil b)



Reserve-Platz auf der folgenden Seite.
(bitte streichen Sie eine ungültige Lösung deutlich)

Aufgabe 4: (PThreads und Parameterübergabe, 15 Punkte [3+2+10])

a) (3 Punkte) Nennen und beschreiben Sie jeweils in einem Satz alle in der Vorlesung besprochenen Arten, wie ein PThread beendet werden kann!

Lösung:

Folie 31: ## andere Foliennr. Im WS14/15

1. Implizit (Rückkehr der Threadfunktion)
2. Explizit via pthread_exit()
3. Cancelled von einem anderen Thread desselben Prozesses

b) (2 Punkte) Nennen Sie kurz die wesentlichen Unterschiede zwischen den beiden Threadzuständen *Joinable* und *Detached*!

Lösung:

Folie 34/35: ## andere Foliennr. Im WS14/15

Joinable: Erlaubt einem anderen Thread, den Rückgabewert zu erhalten; Rückgabewert/Thread ID werden nicht automatisch aufgeräumt

Detached: Nach Beenden des Threads werden Rückgabewert und Thread ID sofort aufgeräumt

c) (10 Punkte) Gegeben sei folgendes Programmfragment (siehe nächste Seite), bei dem die Funktion *main* eine Unterroutine *createThread(int a, int b)* aufruft, welche einen neuen Thread starten und diesem die Werte *a* und *b* übergeben soll. Der Thread, welcher aus der Funktion *workThreadFunction* bestehen soll, soll nichts anderes tun, als die Summe der beiden Werte *a+b* zu berechnen und zurückzugeben. Die *main*-Funktion soll das Ergebnis des Threads ausgeben.

Ergänzen Sie die Signatur der Funktion *workThreadFunction.* Implementieren Sie die Funktionen *workThreadFunction* und *createThread* mit der oben beschriebenen Funktionalität. Ergänzen Sie die Funktion *main* so, dass der Rückgabewert von *workThreadFunction* ausgegeben wird.

Sie können davon ausgehen, dass der neue Thread automatisch im Zustand *Joinable* startet - Sie brauchen keine Threadattribute setzen. Ebenso brauchen Sie sich nicht um eventuelle Fehlerbehandlungen oder Speicherfreigaben zu kümmern. Sie dürfen den vorhandenen Code **nicht** verändern oder außerhalb der von Ihnen zu ergänzenden Funktionen erweitern.

Lösung:

```
pthread_t workThreadID;
struct wtParams {
     int param1, param2;
};


void* workThreadFunction(void *threadArg) {



struct wtParams *wtp;

wtp = (struct wtParams *)threadArg;
return (void *)(wtp->param1 + wtp->param2);



}


void createThread(int a, int b) {

static struct wtParams wtp;

     wtp.param1 = a;
     wtp.param2 = b;
     pthread_create(&workThreadID,                    NULL,
workThreadFunction, &wtp);

Alternative zu "static": malloc()

Wichtig: Parameter für pthread_create dürfen nicht lokal
sein!
}


void main(int argc, char **argv) {
     int threadResult;
     createThread(3, 5);



pthread_join(workThreadID, (void *)&threadResult);

Wichtig: Rückgabe muss nach Beendigung des Threads noch
gültig sein, d.h. nicht lokal in workThreadFunction!


     printf("Resultat: %d\n", threadResult);
}
```

Aufgabe 5: (Synchronisation von Goroutinen, 10 Punkte [6+2+2])

a) (6 Punkte) Schreiben Sie ein Programm welches in der Main-Funktion über eine der Liste der Jobs iteriert deren nebenläufige Verarbeitung durch die Funktion `Do` ausgeführt wird, d.h. die jedes Element soll in einer eigenen Goroutine verarbeitet werden. Die nebenläufige Verarbeitung sollte natürlich thread-safe sein. Um das System nicht zu sehr auszulasten, erlauben dürfen nicht mehr als 16 parallele Instanzen der Funktion `Do` parallel ausgeführt werden. Die Ergebnisse der Verarbeitung in der Funktion Do müssen im Anschluss zurück an die Main-Funktion übergeben werden wo diese dann ausgegeben werden.

b) (2 Punkte) Werden in Ihrem Code zu jeder Routine immer eigene Threads auf der Betriebssystem erstellt (mit Begründung)?

c) (2 Punkte) Was sind Free-Lists?

Aufgabe 6: (Interprozesskommunikation, Pipes Files, 10 Punkte [6 + 4])

a) (6 Punkte) In der Vorlesung wurden diverse Konzepte und Mechanismen zur Interprozesskommunikation (Englisch IPC, Interprocess Communication) besprochen.

Erklären Sie jeweils **kurz** für jedes der unten genannten Persistenz-Schemen die charakteristischen Eigenschaften, und dabei insbesondere, wie lange ein IPC-Objekt im jeweiligen Schema existieren kann:

1. Prozess-Persistenz (Englisch Process Persistence)

Teil a) 6 Punkte, jeweils 2 Pkte, Lösung siehe Kap 3.3. Folie 15

---

wenn der letzte Prozess den Zugriff schließt, ist das IPC-Objekt weg (bsp. Pipe)

---

2. Kernel-Persistenz

existiert bis Kernel-Reboot, oder explizit gelöscht
Bsp.: FIFO

---

3. Dateisystem-Persistenz (Englisch File System Persistence)

überleben auch Reboot, nur wenn explizit gelöscht
Bsp. Files

---

b) (4 Punkte) Nennen Sie, wie ein IPC-Objekt des entsprechenden IPC-Konzeptes von einem Prozess zur Kommunikation eindeutig identifiziert werden kann.

Ggf. helfen Ihnen die Signaturen der entsprechenden Systemfunktionen, siehe die Liste der Syntax relevanter Bibliotheksfunktionen am Ende der Klausur.
Teil b) 4 Punkte, jeweils 1 Punkt

1. Pipes 1 Punkt
durch den Filedeskriptor, vgl. pipe()
Zugriff nur innerhalb des erzeugenden Prozesses und dessen fork()-Kindern

---

2. FIFOs (named Pipes) 1 Punkt
eindeutige Identifikation durch Namen
Zugriff nach open() über Filedeskriptor

---

3. Files 1 Punkt
eindeutige Identifikation durch Namen
Zugriff nach open() über Filedeskriptor

---

4. Sockets (Internet Domain) 1 Punkt
eindeutige Identifikation durch IP-Adresse und Port (ggf. noch durch Protokoll TCP/UDP)
Zugriff nach socket() und ggf. bind(), listen() etc. über Socket-/Filedeskriptor

---

!NICHT ZU BEARBEITEN – nur zur Orientierung!

5. Sockets (Unix Domain)

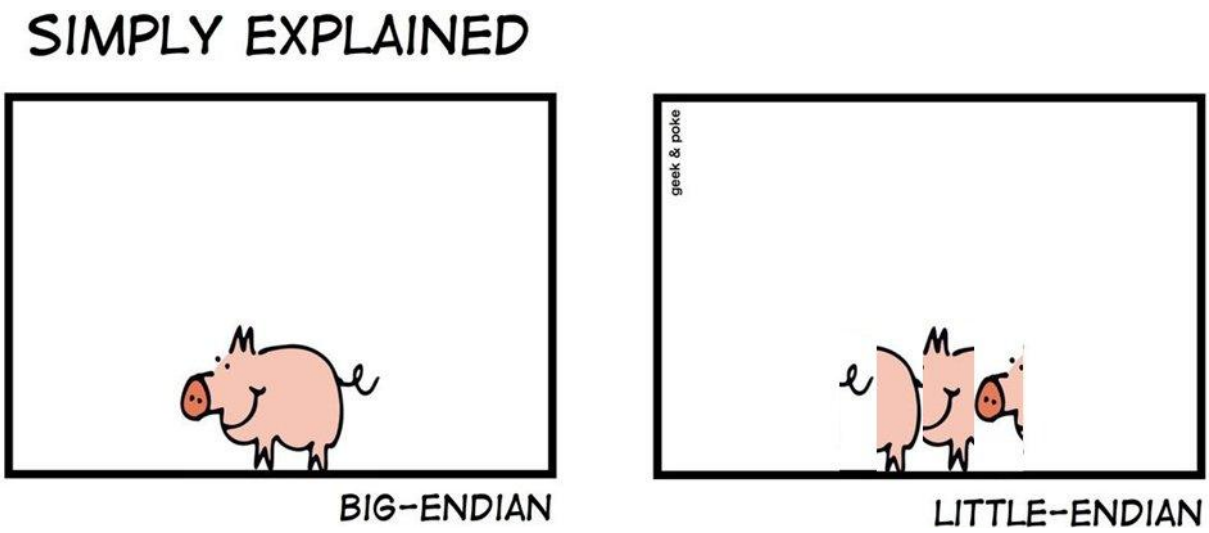
Identifikation über Namen, Zugriff über Socket-/File-Deskriptor

---

6. Message Queues

Identifikation über Namen, Zugriff über MQ-Deskriptor

Aufgabe 7: (Byteorder, 12 Punkte [3+3+6 + 2 Bonuspunkte])



a) (3 Punkte) Erläutern Sie prägnant in maximal 3 Sätzen den oben abgedruckten Comic (er stammt etwas abgeändert von der Webseite geekandpoke.typepad.com)

Es sollte in der Erklärung vorkommen, was die Byteorder bedeutet.

Also zB dass Integerzahlen von einem Typ, der Länger als 1 Byte ist (16 Bit, 32 Bit, 64 Bit) in der einen oder anderen Reihenfolge byte-weise im Speicher abgelegt werden.

Stichworte Byte-Order für eine CPU-Architektur,
Network Byte Order für die koordinierte Übertragung über ein Netzwerk, zwischen Rechnern mit möglicherweise unterschiedlicher (Host) Byte-Order.

b) (3 Punkte) Handelt es sich bei dem abgebildeten Tier um ein 8-Bit-, ein 16-Bit- oder ein 32-Bit-Schwein? **Erläutern Sie ihre Antwort.**

32 Bit

Das Schwein wird in vier Teile zerteilt. Ein Teil = 1 Byte, also 4 Byte = 32 Bit.

c) (6 reguläre Punkte) Geben Sie ein C/C++-Codefragment an, mit dem Sie testen könnten, ob das Programm auf einer Big-Endian-CPU oder einer Little-Endian-CPU läuft. Hier kommt es uns darauf an, dass Sie eine möglichst effiziente/elegante Lösung mit wenigen Instruktionen finden. **Bitte kommentieren Sie Ihre Lösung ausreichend.**

Für eine möglichst kurze Lösung gibt es **bis zu 2 Bonus-Punkte!**

kürzest mögliche Lösung nutzt die Systemfunktion htonl() oder ntohl(). Die Big Endian Byte Order ist identisch zur Network Byte Order, somit ändern die Funktionen htonl() oder ntohl() auf einem Big Endian System nichts (Vergleich ist WAHR).

2 Bonus-Punkte, wenn diese kurze Lösung gewählt wurde.

```
if ( htonl(47) == 47 ) {
  // Big endian
} else {
  // Little endian.
}
```

- Alternativ: -die anderen Lösungen sind vermutlich länger

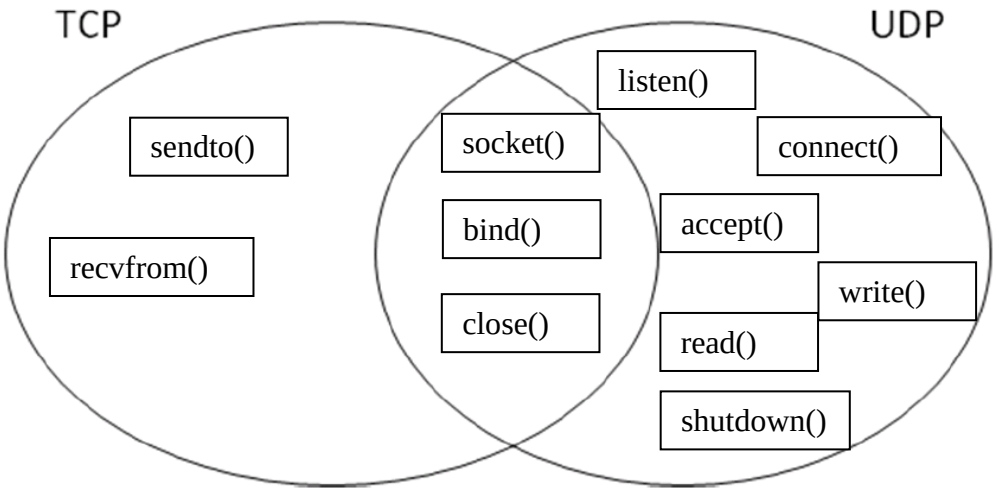
```
#define BIG_ENDIAN      0
#define LITTLE_ENDIAN   1

int TestByteOrder()
{
   short int word = 0x0001;
   char *byte = (char *) &word;
   return(byte[0] ? LITTLE_ENDIAN : BIG_ENDIAN);
      }
```

- Es sind andere Varianten denkbar, die Werte von 2 oder 4 Byte langen Variablen geeignet abfragen.

Aufgabe 8: (Sockets in C, 10 Punkte [11x0,5+9x0,5])

a) Ordnen Sie die folgenden Funktionsnamen zum Umgang mit Sockets den Protokollen UDP und TCP zu, indem Sie sie in den jeweiligen Mengenkreis eintragen. Ordnen Sie die Funktionen, die für beide Protokolle Verwendung finden, dem Schnittbereich der beiden Mengen zu. ### Zuordnung hier ist genau falsch herum UDP <-> TCP



Die zuzuordnenden Funktionsnamen sind: accept(), bind(), close(), connect(), listen(), read(), recvfrom(), sendto(), shutdown(), socket(), write().

b) Ergänzen Sie das angegebene allgemeine Ablaufschema einer UDP Client/Server Kommunikation über Sockets mit den entsprechenden C-Funktionsaufrufen. Vernachlässigen Sie dabei die Parameter der Funktionen.

